

Cross-Entropy, Deeper

What the loss number really measures · video 2:32

This lesson answers one question: what exactly is the “loss” every training run prints? It is the average of **$-\ln(\text{probability of the right token})$** . The same number can be read as **perplexity** (e^{loss}) or as **bits** ($\text{loss} \div \ln 2$). The log punishes confident mistakes hard, rewards honest probabilities (GPT-2 is well calibrated, except for one formatting habit), and its gradient is beautifully simple: softmax minus one-hot.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 6).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The loss, token by token	▶ 0:08-0:47	2	☐
2. Perplexity, bits, and confident mistakes	▶ 0:47-1:27	3	☐
3. Calibration	▶ 1:27-1:56	2	☐
4. The gradient, and the code	▶ 1:56-2:23	2	☐

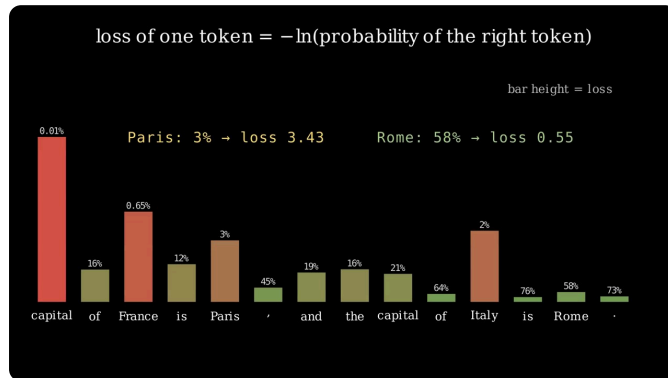
Before you start: How LLMs Work episode 10 (training) and Foundations (logarithms, softmax).
Code: `code/d17_cross_entropy` (GPT-2 small, about 500 MB).

CONCEPT 1 OF 4

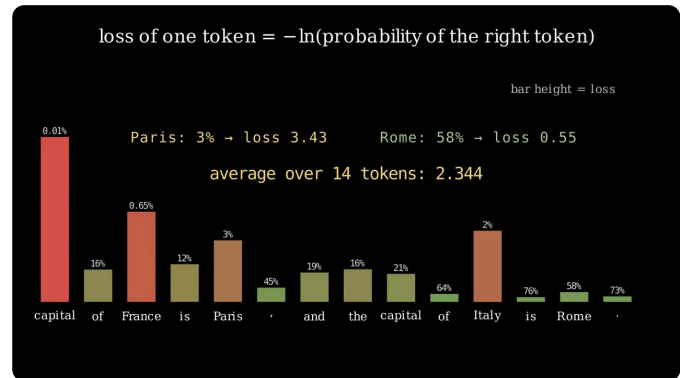
The loss, token by token

▶ Video 0:08–0:47

The loss of one token is $-\ln(p)$, where p is the probability the model gave to the token that actually came next. GPT-2 on “The capital of France is Paris, and the capital of Italy is Rome.”: after “The capital of France is”, “Paris” gets **3%** (loss 3.43). By the second sentence the model has caught on: “Rome” gets **58%** (loss 0.55), and “ of” after “ capital” 64% (0.45).



▶ 0:40 GPT-2 on two sentences: “Paris” gets 3% (loss 3.43); by the second sentence “Rome” gets 58% (0.55).



▶ 0:46 The printed loss is the average over the tokens: 2.344.

The number training prints is the **average** over all predicted tokens: 2.344 for this sentence (14 predictions).

KEY IDEA

Cross-entropy = average of $-\ln(\text{probability of the right token})$. Lower is better; 0 means certainty and correctness.

Check 1: The loss, token by token

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 CALCULATION

The model gives the right token 25%. What is its loss?

1.2 TRUE OR FALSE

“A loss of 0 is possible only if the model gives the right token 100%.”

True ☐ False ☐

CONCEPT 2 OF 4

Perplexity, bits, and confident mistakes

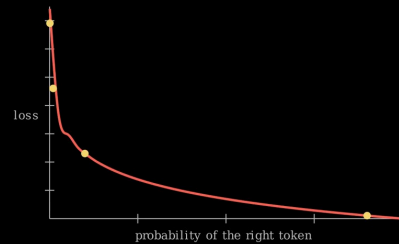
▶ Video 0:47-1:27

Two other names for the same number. **Perplexity** = $e^{\text{loss}} = e^{2.344} = \mathbf{10.4}$: as if the model were choosing evenly among about 10 tokens at each step. **Bits** = $\text{loss} \div \ln 2 = \mathbf{3.38}$ bits per token. On 1,024 tokens of Shakespeare GPT-2's loss is 4.000 (perplexity 54.6), which is **1.87 bits per character**.

perplexity = $e^{\text{loss}} = e^{2.344} = 10.4$
like choosing evenly among ~10 tokens

bits = $\text{loss} \div \ln 2 = 3.38$ bits per token

Shakespeare (1,024 tokens): 1.87 bits per character



▶ 1:06 Perplexity $e^{2.344} = 10.4$; 3.38 bits per token; Shakespeare: 1.87 bits per character.

▶ 1:25 $-\ln p$ explodes near zero: 90% → 0.11, 10% → 2.30, 1% → 4.61, 0.1% → 6.91.

The log punishes confident mistakes hard: right token at 90% → 0.11, 10% → 2.30, 1% → 4.61, 0.1% → 6.91. On Shakespeare, the worst 10% of tokens make up **32%** of the total loss.

KEY IDEA

Perplexity and bits are the same measurement in different units; a few very wrong predictions can dominate the average.

Check 2: Perplexity, bits, and confident mistakes

Answer all 3 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

A model's loss is 1.0. What are its perplexity and its bits per token?

2.2 CALCULATION

Perplexity 54.6 on Shakespeare. What loss is that?

2.3 SHORT ANSWER

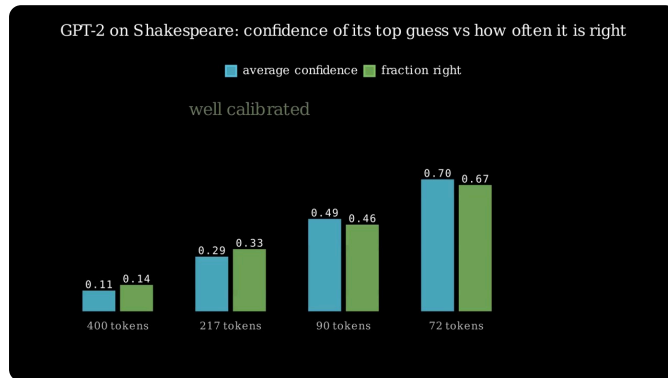
Why does a model that is wrong but unsure lose less than one that is wrong and sure?

CONCEPT 3 OF 4

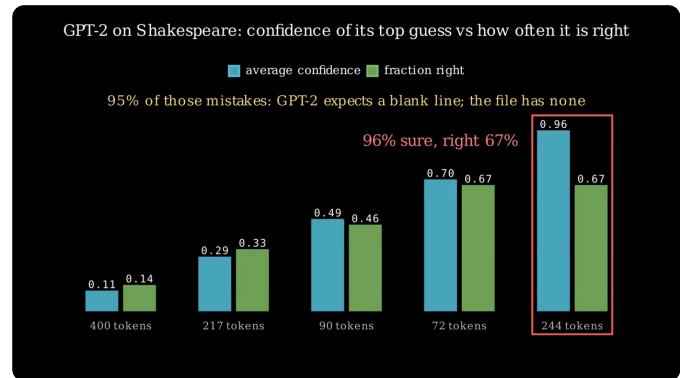
Calibration

▶ Video 1:27-1:56

Because the log rewards honest probabilities, a model trained on cross-entropy tends to be **calibrated**: when its top guess has about 30% probability, GPT-2 is right 33% of the time; around 70%, right 67%. Buckets on 1,023 Shakespeare tokens: confidence 0.11 → right 0.14, 0.29 → 0.33, 0.49 → 0.46, 0.70 → 0.67.



▶ 1:39 When GPT-2's top guess has ~30% probability, it is right 33% of the time; ~70%, right 67%.



▶ 1:55 At 96% confidence it is right only 67%: 95% of those mistakes are a line break the file doesn't have.

Except at the top: in the 244 tokens where GPT-2 is on average **96%** sure, it is right only **67%** of the time. Of the 81 confident mistakes, 95% are the same one: GPT-2 expects a blank line after each line of Shakespeare (a line break token), and this file has none. A mismatch between the text and what the model is used to, not random overconfidence.

KEY IDEA

Cross-entropy rewards calibrated probabilities; systematic miscalibration often points to a mismatch between the data and what the model expects.

Check 3: Calibration

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

A perfectly calibrated model says 40% for its top guess on 500 tokens. About how many of those guesses are right?

- A. 50
- B. 200
- C. 400
- D. 500

3.2 TRUE OR FALSE

"GPT-2 was overconfident on Shakespeare at every confidence level."

True ☐ False ☐

CONCEPT 4 OF 4

The gradient, and the code

▶ Video 1:56–2:23

For each logit, the gradient of the loss is the predicted probability, minus 1 for the right token:

softmax – one-hot. With 5 logits and the right token at index 2: [+0.612, +0.098, –0.985, +0.231, +0.044], exactly what autograd computes. The right token's logit is pushed up, every other logit down in proportion to its probability. That is the signal that starts every backward pass.

gradient of the loss w.r.t. the logits (right token: index 2)

softmax	+0.076	+0.209	+0.129	+0.054	+0.532
– one-hot	0	0	1	0	0
= gradient	+0.076	+0.209	–0.871	+0.054	+0.532
autograd	+0.076	+0.209	–0.871	+0.054	+0.532

the signal that starts every backward pass

▶ 2:12 Gradient w.r.t. the logits = softmax – one-hot; autograd agrees to every digit shown.

```
1 logp = logits.log_softmax(dim=-1) # (T, vocabulary)
2 loss = -logp[torch.arange(T), targets].mean() # pick the right token, negate, average
3
4 # the same thing, built in:
5 loss = F.cross_entropy(logits, targets)
```

▶ 2:22 log_softmax, pick the right token, negate, average: what F.cross_entropy does.

KEY IDEA

$\partial \text{loss} / \partial \text{logit}_i = p_i - 1[i = \text{right}]$: simple, bounded between –1 and 1, and cheap to compute.

Check 4: The gradient, and the code

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

Softmax gives the right token 0.7. What is the gradient for its logit? For a wrong token with 0.2?

4.2 HANDS-ON CODE

Try it yourself. Check that the gradient sums to zero over the logits, for any input.

```
z = torch.randn(5, requires_grad=True)
F.cross_entropy(z[None], torch.tensor([2])).backward()
print(z.grad.sum())
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The loss, token by token · rewatch ▶ 0:08-0:47

1.1 About 1.39. $-\ln 0.25 = \ln 4 \approx 1.386$.

1.2 True. $-\ln p = 0$ only when $p = 1$.

2. Perplexity, bits, and confident mistakes · rewatch ▶ 0:47-1:27

2.1 Perplexity about 2.72; about 1.44 bits. $e^1 \approx 2.718$; $1 / \ln 2 \approx 1.443$.

2.2 4.0. $\ln 54.6 \approx 4.000$, matching the measured loss.

2.3 The loss depends only on the probability given to the right token; an unsure model leaves more probability for it, so $-\ln p$ is smaller. Confidence in the wrong answer means little probability left for the right one.

3. Calibration · rewatch ▶ 1:27-1:56

3.1 B. 40% of 500 = 200.

3.2 False. Below 0.8 it was close to calibrated (even slightly underconfident at $0.11 \rightarrow 0.14$ and $0.29 \rightarrow 0.33$); only the top bucket was far off.

4. The gradient, and the code · rewatch ▶ 1:56-2:23

4.1 -0.3 and +0.2. $0.7 - 1$ and $0.2 - 0$.

4.2 0 (up to rounding, about 10^{-8}). Softmax sums to 1 and the one-hot sums to 1, so their difference sums to 0.